



ACTIVATION FUNCTION

Deep Learning

Activation Function

- **An Activation Function** decides whether a neuron should be activated or not.
- An activation function is a function that is added into an artificial neural network in order to help the **network learn complex patterns in the data**.
- When comparing with a neuron-based model that is in our brains, the activation function is at the end deciding **what is to be fired to the next neuron**.
- That is exactly what an activation function does in an ANN as well.
- **It takes in the output signal from the previous cell and converts it into some form that can be taken as input to the next cell.**

Need for Activation Function

- The purpose of an activation function is to add **non-linearity** to the neural network.
- A neural network without an activation function is essentially just a **linear regression** model.
- Most **real-world data is non-linear**.
- For example, relationships between house prices and size, income, and purchases, etc., are non-linear. If neural networks had no activation functions, they would fail to learn the complex non-linear patterns that exist in real-world data.
- Activation functions enable neural networks to learn these non-linear relationships by introducing non-linear behaviors through activation functions. This greatly increases the flexibility and power of neural networks to model complex and nuanced data.

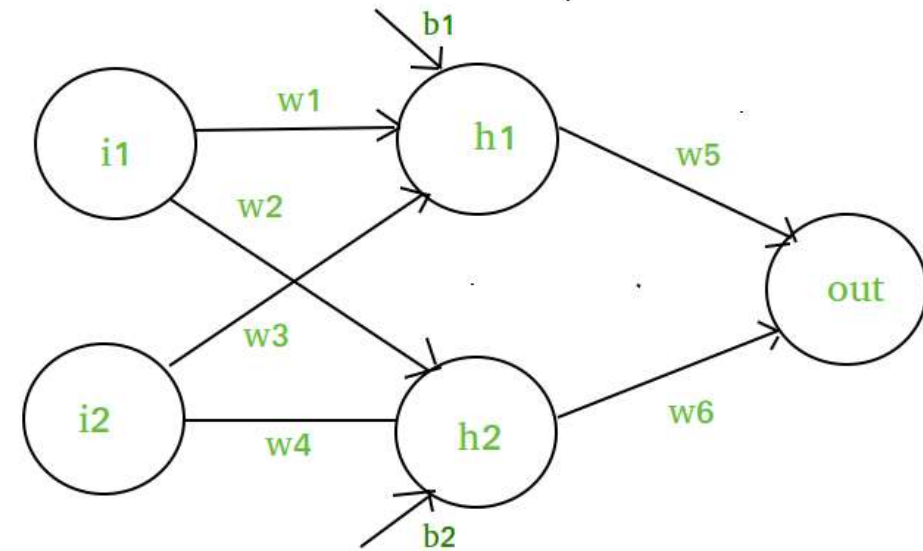
Need for Activation Function

Mathematical proof:

- Hidden layer i.e. layer 1:

$$z(1) = W(1)X + b(1) = a(1)$$

- Here,
- $z(1)$ is the vectorized output of layer 1
- $W(1)$ be the vectorized weights assigned to neurons of hidden layer i.e. w_1, w_2, w_3 and w_4
- X be the vectorized input features i.e. i_1 and i_2
- b is the vectorized bias assigned to neurons in hidden layer i.e. b_1 and b_2
- $a(1)$ is the vectorized form of any linear function.

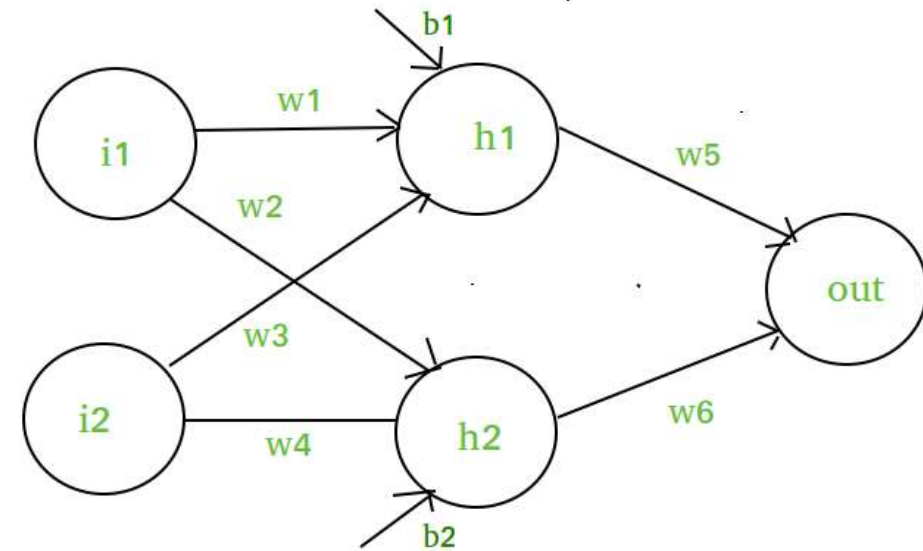


(Note: We are not considering activation function here)

Need for Activation Function

Mathematical proof:

- Layer 2 i.e. output layer :-
- $z(2) = W(2)a(1) + b(2)$
- $a(2) = z(2)$

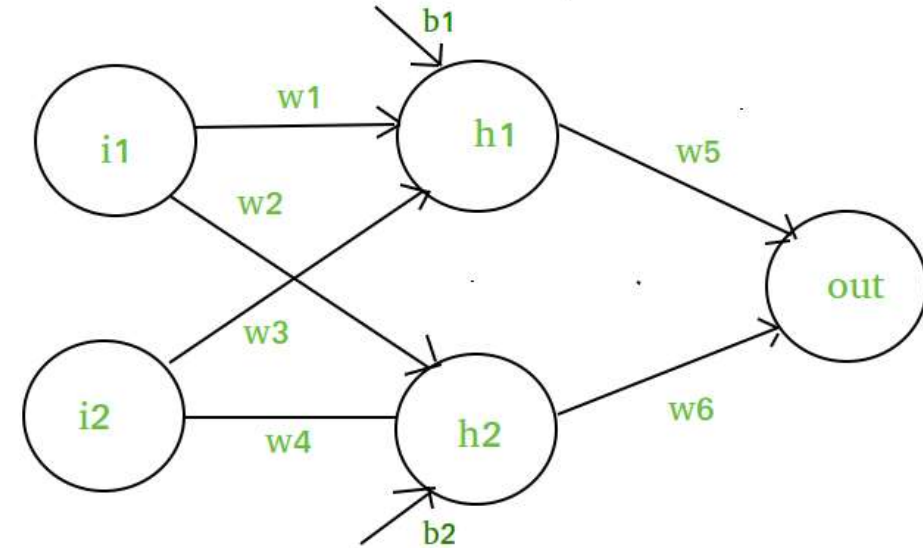


(Note : Input for layer 2 is output from layer 1)

Need for Activation Function

Mathematical proof:

- Calculation at Output layer:-
- $z(2) = (W(2) * [W(1)X + b(1)]) + b(2)$
- $z(2) = [W(2) * W(1)] * X + [W(2)*b(1) + b(2)]$
- Let, $W(2) * W(1) = W$
- $[W(2)*b(1) + b(2)] = b$
- Final output : $z(2) = W*X + b$
- which is again a linear function



Need for Activation Function

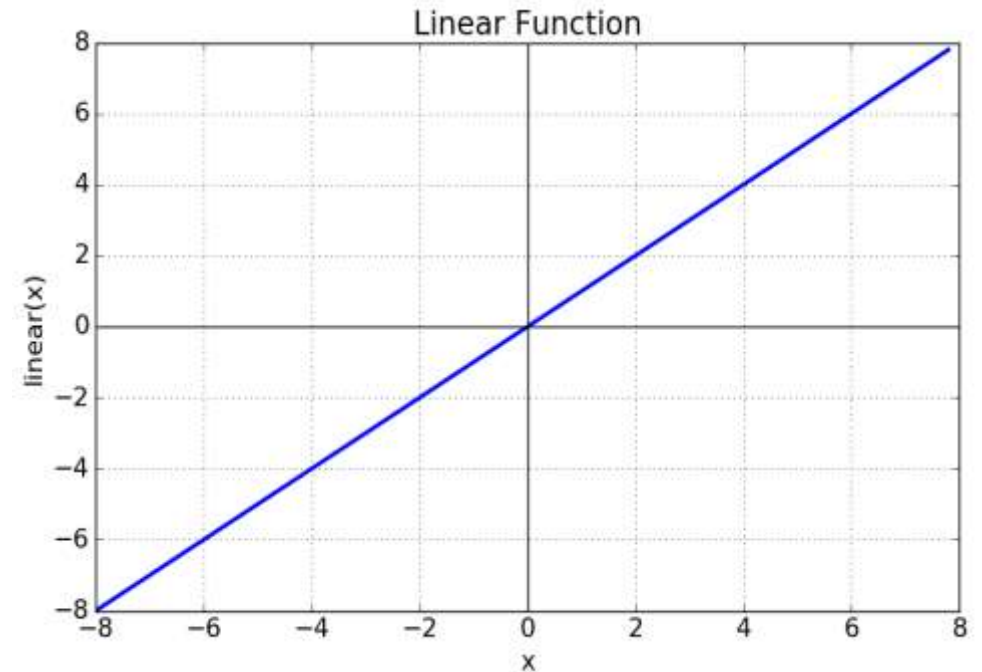
- This observation results again in a linear function even after applying a hidden layer, hence we can conclude that, doesn't matter how many hidden layer we attach in neural net, all layers will behave same way because the **composition of two linear function is a linear function itself**.
- Neuron can not learn with just a linear function attached to it.
- A non-linear activation function will let it learn as per the difference w.r.t error.
- **Hence we need an activation function.**

Types of Activation Function

- Activation function is used to determine the output of neural network like yes or no.
- It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).
- The Activation Functions can be basically divided into 2 types-
 1. Linear Activation Function
 2. Non-linear Activation Functions

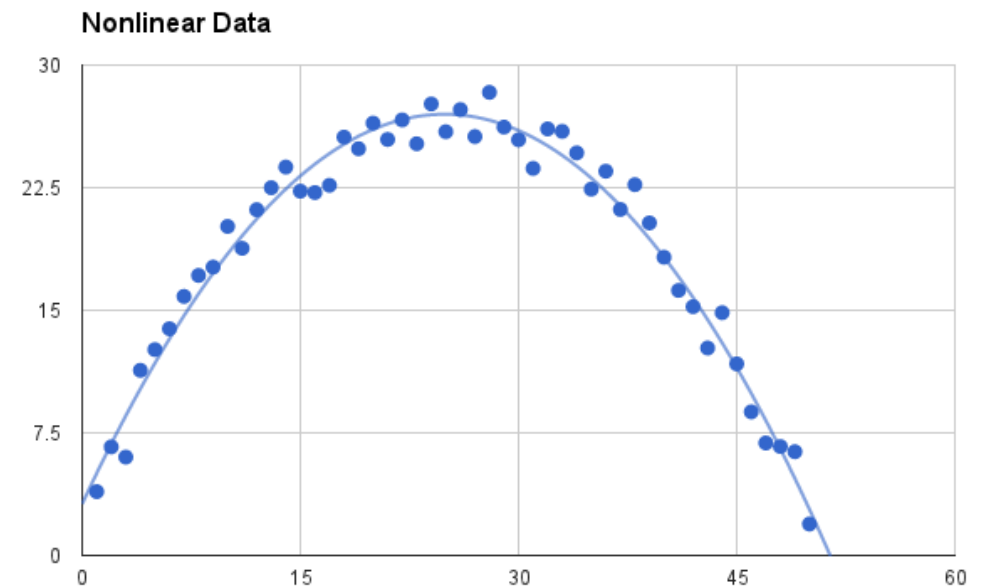
Linear Activation Function

- Also known as "no activation," or "identity function" (multiplied $\times 1.0$), is where the activation is proportional to the input.
- The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.
- **Equation : $f(x) = x$**
- **Range : (-infinity to infinity)**
- It doesn't help with the complexity or various parameters of usual data that is fed to the neural networks.



Non-Linear Activation Function

- It solve the following limitations of linear activation functions:
 - Any output can be represented as a functional computation in a neural network.
- **Allow backpropagation** because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- **Allow the stacking of multiple layers** of neurons as the output would now be a non-linear combination of input passed through multiple layers.



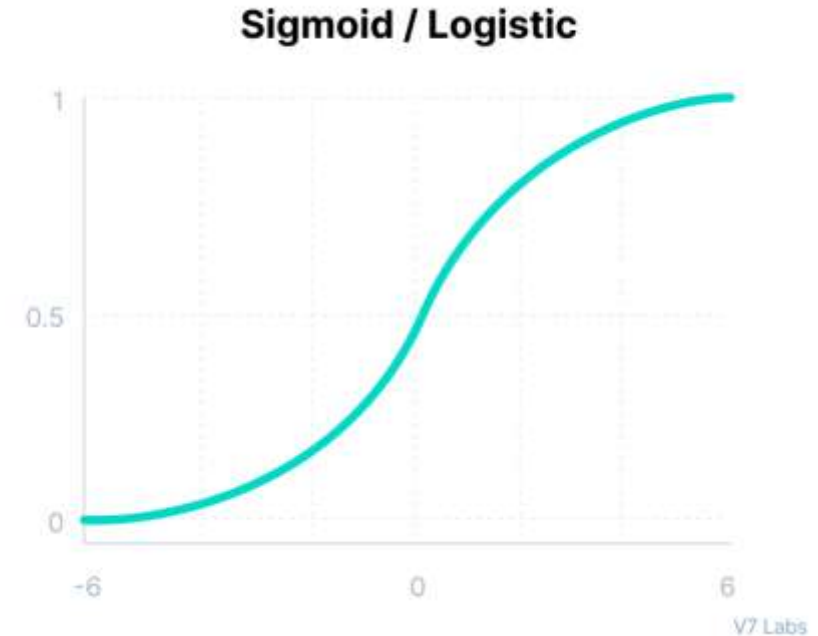
Types of Non-Linear Activation Function

Sigmoid / Logistic Activation Function

- This function takes any **real value as input** and **outputs values in the range of 0 to 1**.
- The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0.
- Mathematically it can be represented as:

Sigmoid / Logistic

$$f(x) = \frac{1}{1 + e^{-x}}$$



Sigmoid / Logistic Activation Function

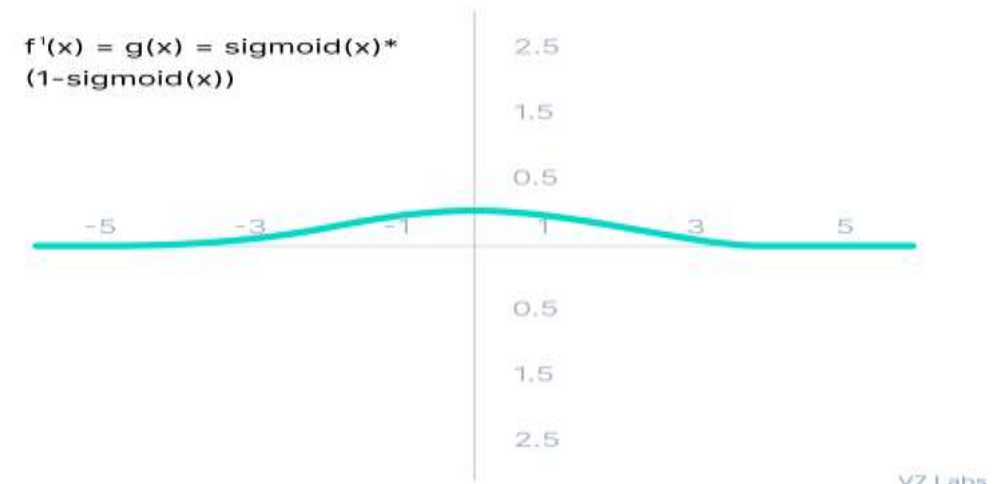
- The sigmoid/logistic activation function is one of the most widely used functions:
- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

Why derivative/differentiation is used ?

- When updating the curve, to know in **which direction** and **how much** to change or update the curve depending upon the slope.
- That is why we use differentiation in almost every part of Machine Learning and Deep Learning.

Sigmoid / Logistic Activation Function

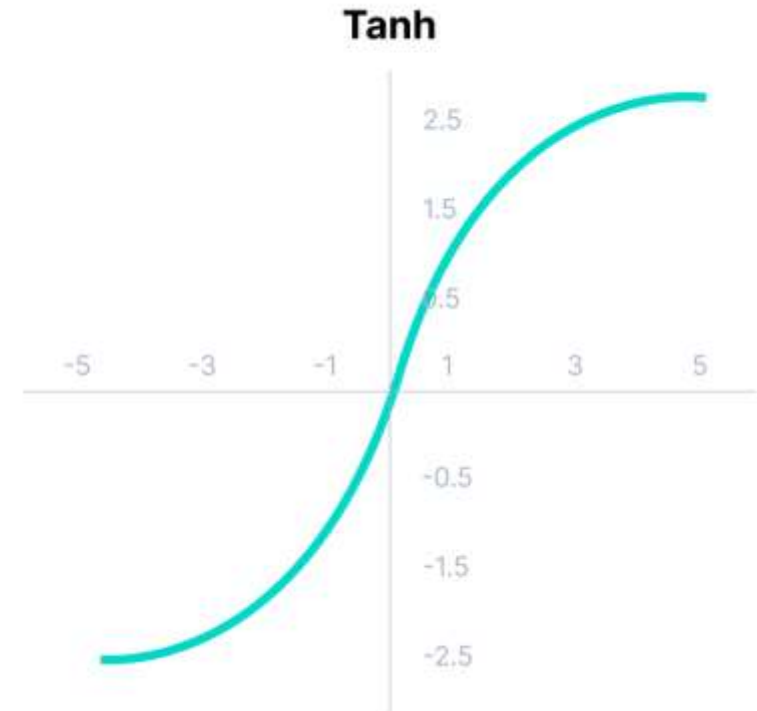
- The limitations of sigmoid function:
- The derivative of the function is $f'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$.
- The gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.
- It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the Vanishing gradient problem.
- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.



Tanh Function (Hyperbolic Tangent)

- Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1.
- In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.
- Mathematically it can be represented as;

$$\text{Tanh}$$
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

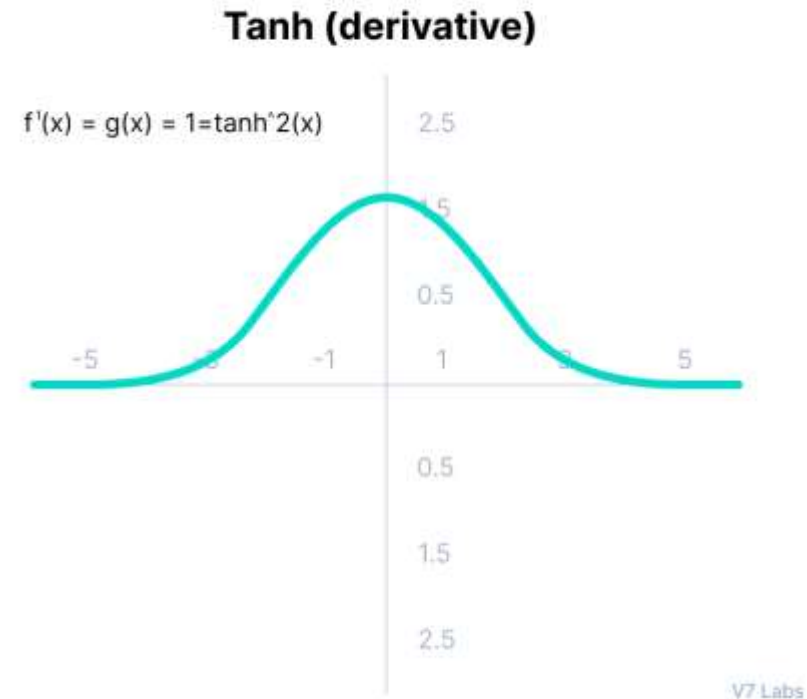


Tanh Function (Hyperbolic Tangent)

- **Advantages** of using this activation function are:
- The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- Usually used in hidden layers of a neural network as its values lie between -1 to 0; therefore, the mean for the hidden layer comes out to be 0 or very close to it.
- It helps in centering the data and makes learning for the next layer much easier.

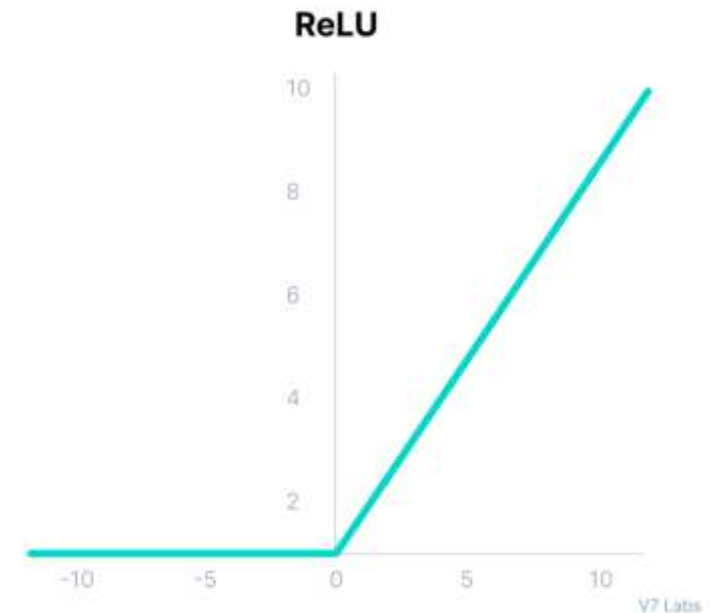
Tanh Function (Hyperbolic Tangent)

- it also faces the problem of vanishing gradients similar to the sigmoid activation function.
- Plus the gradient of the tanh function is much steeper as compared to the sigmoid function.
- tanh is zero centered, and the gradients are not restricted to move in a certain direction. Therefore, in practice, tanh nonlinearity is always preferred to sigmoid nonlinearity.



ReLU Function

- ReLU stands for **Rectified Linear Unit**.
- Mathematically it can be represented as:
$$ReLU$$
$$f(x) = \max(0, x)$$
- Although it gives an impression of a linear function, ReLU has a derivative function and **allows for backpropagation** while simultaneously making it computationally efficient.
- ReLU function does not activate all the neurons at the same time.
- The neurons will only be deactivated if the output of the linear transformation is less than 0.

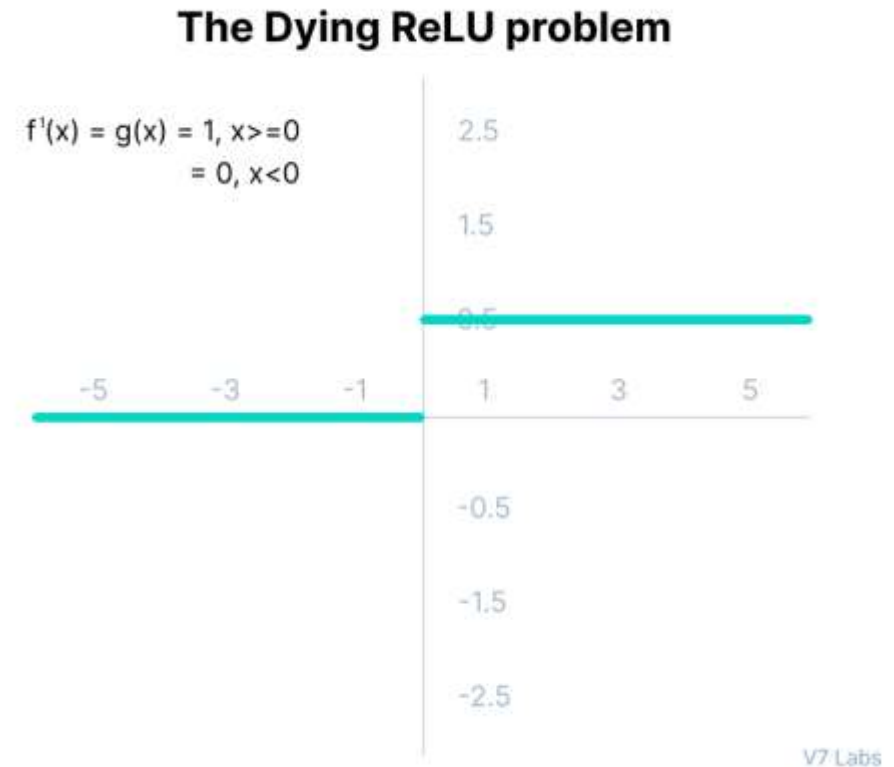


ReLU Function

- The **advantages** of using ReLU as an activation function are as follows:
- Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.
- ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

ReLU Function

- The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can create **dead neurons** which never get activated.
- All the negative input values become zero immediately, which decreases the model's ability to fit or train from the data properly.

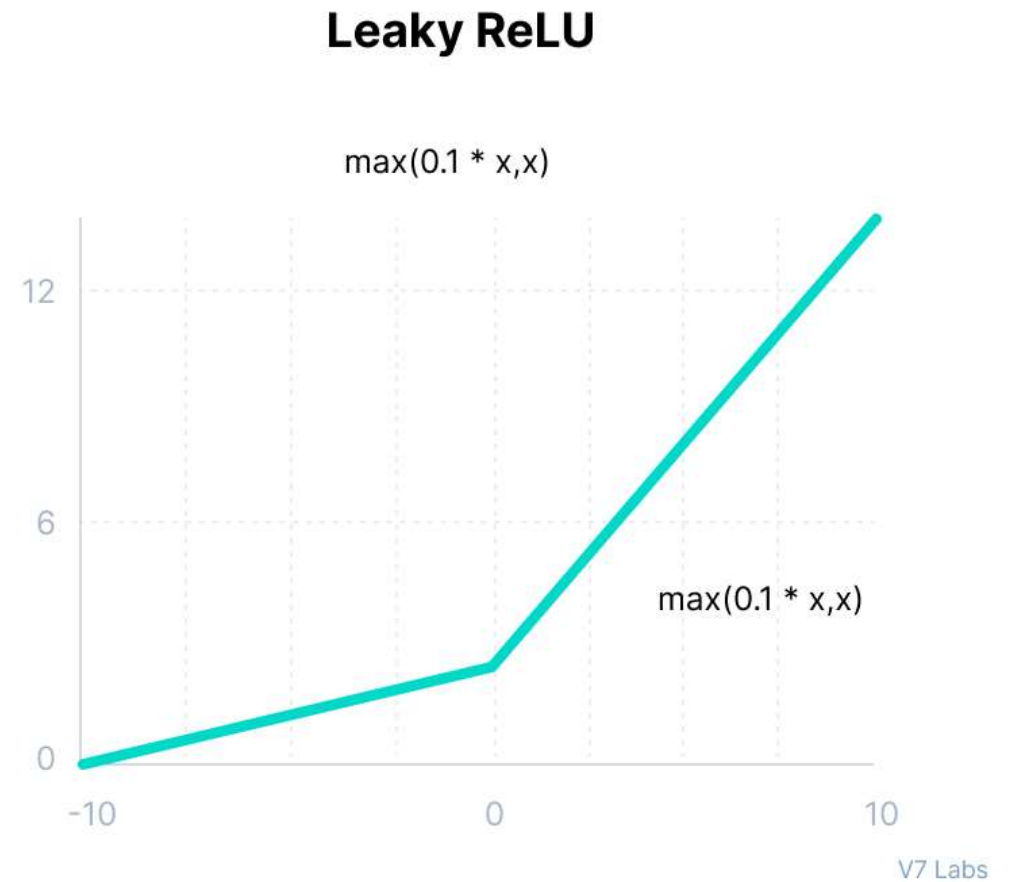


Leaky ReLU Function

- Leaky ReLU is an improved version of ReLU function to solve the Dying ReLU problem as it has a small positive slope in the negative area.
- It enable backpropagation, even for negative input values.
- Mathematically it can be represented as:

Leaky ReLU

$$f(x) = \max(0.1x, x)$$

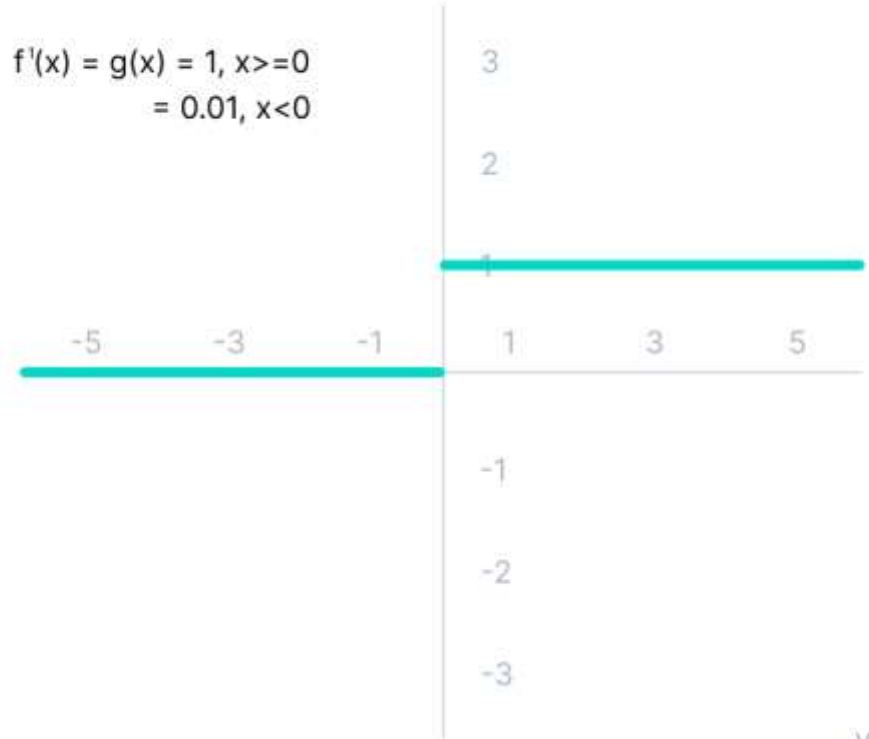


Leaky ReLU Function

- The **limitations** that this function faces include:
- The predictions may not be consistent for negative input values.
- The gradient for negative values is a small value that makes the learning of model parameters time-consuming.

Leaky ReLu (derivative)

$$f'(x) = g(x) = 1, x \geq 0 \\ = 0.01, x < 0$$

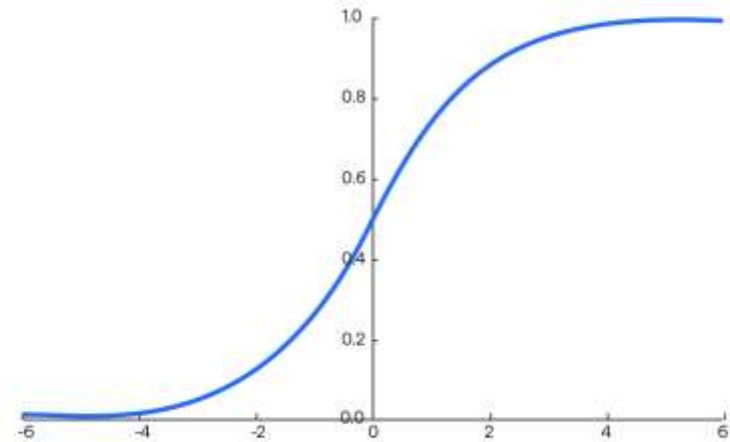


Softmax Function

- The softmax activation function forces the values of output neurons to take values between zero and one, so they can represent probability scores.
- Mathematical representation:

$$y_i = \frac{e^{z_i}}{\sum_{i=0}^m e^{z_i}}$$

Softmax Function



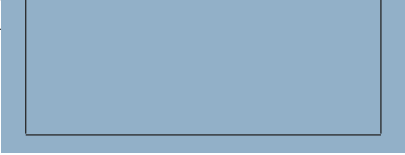
How to choose activation function?

- You need to match your activation function for your output layer based on the type of prediction problem that you are solving—specifically, the type of predicted variable.
- As a **rule of thumb**, you can begin with using the ReLU activation function and then move over to other activation functions if ReLU doesn't provide optimum results.
- For output layer;
 1. **Regression** - Linear Activation Function
 2. **Binary Classification**—Sigmoid/Logistic Activation Function
 3. **Multiclass Classification**—Softmax
 4. **Multilabel Classification**—Sigmoid

How to choose activation function?

- For Hidden layers ;

1. **Convolutional Neural Network (CNN):** ReLU activation function.
2. **Recurrent Neural Network:** Tanh and/or Sigmoid activation function.



THANK YOU!!